```
import numpy as np
```

```
np.full((2,3),3)
```

```
array([[3, 3, 3],
       [3, 3, 3]])
```

```
np.empty((2,2))
```

```
array([[2.17226034e-315, 0.00000000e+000],
       [1.35717430e+131, 6.40695298e+029]])
```

```
np.linspace(0,10,4)
```

```
array([ 0.        ,  3.33333333,  6.66666667, 10.        ])
```

```
np.arange(2,8)
np.arange(2,8,2)
```

```
array([2, 4, 6])
```

```
a = np.array([[1, 2], [3, 4]])

a.reshape(4)      # [1, 2, 3, 4]
a.flatten()       # [1, 2, 3, 4]
a.ravel()         # [1, 2, 3, 4]
a.transpose()     # [[1, 3], [2, 4]]
```

```
array([[1, 3],
       [2, 4]])
```

```
import numpy as np

a = np.array([[10, 20, 30],
              [40, 50, 60]])

print(a.ndim)
print(a.shape)
print(a.size)
```

```
2
(2, 3)
6
```

```
import numpy as np

a = np.array([1.5, 2.5, 3.5, 4.5])

print(a.ndim)
print(a.shape)
print(a.size)
print(a.dtype)
print(a.itemsize)
```

```
1
(4,)
4
float64
8
```

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]], dtype=np.int32)

print(a.ndim)
print(a.shape)
print(a.size)
print(a.dtype)
print(a.itemsize)
```

```
2
(2, 3)
6
int32
4
```

```
import numpy as np

a = np.array([[1, 2], [3, 4], [5, 6]])

b = a.reshape(2, 3)

print(b.ndim)
print(b.shape)
print(b.size)
```

```
2
(2, 3)
6
```

```
import numpy as np

print(np.zeros((2, 2)))
print(np.ones((2, 3)))
print(np.full((2, 2), 7))
```

```
[[0. 0.]
 [0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]]
[[7 7]
 [7 7]]
```

```
import numpy as np

a = np.arange(2, 11, 2)
b = np.linspace(2, 10, 5)

print(a)
print(b)
print(a.dtype)
print(b.dtype)
```

```
[ 2  4  6  8 10]
[ 2.  4.  6.  8. 10.]
int64
float64
```

```
import numpy as np

a = np.empty((2, 2))
b = np.full((2, 2), 5)

print(a.shape)
print(b)
```

```
(2, 2)
[[5 5]
 [5 5]]
```

```
import numpy as np

a = np.array([10, 20, 30, 40, 50, 60, 70])

print(a[2:6])
print(a[-4:])
print(a[::2])
print(a[::-1])
```

```
[30 40 50 60]
[40 50 60 70]
[10 30 50 70]
[70 60 50 40 30 20 10]
```

```
import numpy as np

a = np.arange(1, 17).reshape(4, 4)

print(a[1:3, 0:2])
print(a[:, -1])
print(a[::2, ::2])
```

```
[[ 5  6]
 [ 9 10]]
[ 4  8 12 16]
[[ 1  3]
 [ 9 11]]
```

```
import numpy as np

a = np.arange(1, 7).reshape(2, 3)

print(a.flatten())
print(a.ravel())
print(a.transpose())
```

```
[1 2 3 4 5 6]
[1 2 3 4 5 6]
[[1 4]
 [2 5]
 [3 6]]
```

```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = a.transpose()
c = a.reshape(3, 2)

print(b)
print(b.shape)
print(c)
print(c.shape)
```

```
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
[[1 2]
 [3 4]
 [5 6]]
(3, 2)
```

```
import numpy as np

a = np.array([[1, 2], [3, 4]])

b = a.flatten()
c = a.ravel()

b[0] = 100
c[1] = 200

print(a)
print(b)
print(c)
```

```
[[  1 200]
 [  3   4]]
[100   2   3   4]
[  1 200   3   4]
```

```
import numpy as np

a = np.arange(1, 13, dtype=np.int32)
b = a.reshape(3, 4)

print(b.shape)
print(b.size)
print(b.itemsize)
print(b[1:, ::2])
```

```
(3, 4)
12
4
[[ 5  7]
 [ 9 11]]
```

```
import numpy as np

a = np.arange(1, 21)
b = a.reshape(4, 5)

print(b[1:4:2, 1::2])
print(b[::-1, 0])
```

```
[[ 7  9]
 [17 19]]
[16 11  6  1]
```

```
import numpy as np

a = np.linspace(0, 10, 6)
b = a.astype(int)
c = b.reshape(2, 3)

print(a)
print(b)
print(c)
print(c.dtype)
```

```
[ 0.  2.  4.  6.  8. 10.]
[ 0  2  4  6  8 10]
[[ 0  2  4]
 [ 6  8 10]]
int64
```

```
import numpy as np

a = np.arange(1, 13).reshape(3, 4)

b = a[:, 1:3]
c = b.flatten()

print(b)
print(b.shape)
print(c)
print(c.shape)
```

```
[[ 2  3]
 [ 6  7]
 [10 11]]
(3, 2)
[ 2  3  6  7 10 11]
(6,)
```

```
import numpy as np

a = np.array([[2, 4, 6], [8, 10, 12]])

print(a.ndim)
print(a.shape)
print(a.size)
```

```
2
(2, 3)
6
```

```
a = np.array([1.5, 2.5, 3.5], dtype=np.float32)

print(a.dtype)
print(a.itemsize)
print(a.size)
```

```
float32
4
3
```

```
a = np.arange(1, 13)
b = a.reshape(3, 4)

print(b.ndim)
print(b.shape)
print(b.size)
print(b.itemsize)
```

```
2
(3, 4)
12
8
```

```
print(np.zeros((2, 3)))
print(np.ones((2, 2)))
print(np.full((2, 2), 7))
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
[[1. 1.]
 [1. 1.]]
[[7 7]
 [7 7]]
```

```
a = np.arange(1, 10, 2)
b = np.linspace(1, 9, 5)

print(a)
print(b)
```

```
[1 3 5 7 9]
[1. 3. 5. 7. 9.]
```

```
a = np.zeros((2, 2), dtype=int)
b = np.full((2, 2), 3.5)
c = np.linspace(0, 1, 3)

print(a.dtype)
print(b)
print(c)
```

```
int64
[[3.5 3.5]
 [3.5 3.5]]
[0.  0.5 1. ]
```

```
a = np.array([10, 20, 30, 40, 50, 60, 70])

print(a[1:5])
print(a[:3])
print(a[4:])
```

```
[20 30 40 50]
[10 20 30]
[50 60 70]
```

```
a = np.array([5, 10, 15, 20, 25, 30, 35, 40])

print(a[-3])
print(a[::2])
print(a[::-1])
```

```
30
[ 5 15 25 35]
[40 35 30 25 20 15 10  5]
```

```
a = np.arange(1, 17).reshape(4, 4)

print(a[1, :])
print(a[:, 2])
print(a[1:3, 1:3])
```

```
[5 6 7 8]
[ 3  7 11 15]
[[ 6  7]
 [10 11]]
```

```
a = np.arange(1, 26).reshape(5, 5)

print(a[4:0:-2, 4:0:-2])
```

```
[[25 23]
 [15 13]]
```

```
a = np.arange(1, 21).reshape(4, 5)

print(a[::2, 1::2])
print(a[::-1, -1])
```

```
[[ 2  4]
 [12 14]]
[20 15 10  5]
```

```
a = np.arange(1, 7)
b = a.reshape(2, 3)

print(b)
print(b.shape)
```

```
[[1 2 3]
 [4 5 6]]
(2, 3)
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

b = a.transpose()

print(b)
print(b.shape)
```

```
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

```
a = np.array([[1, 2], [3, 4]])

b = a.flatten()
c = a.ravel()

b[0] = 100
c[1] = 200

print(a)
```

```
[[  1 200]
 [  3   4]]
```

```
a = np.arange(2, 14, dtype=np.int32).reshape(3, 4)
b = a[1:, ::2]

print(a.shape)
print(a.size)
print(a.itemsize)
print(b)
print(b.shape)
```

```
(3, 4)
12
4
[[ 6  8]
 [10 12]]
(2, 2)
```

```
a = np.arange(1, 13)
b = a.reshape(3, 4)
c = b.transpose()

print(c)
print(c[1:3, ::2])
print(c[1:3, ::2].shape)
```

```
[[ 1  5  9]
 [ 2  6 10]
 [ 3  7 11]
 [ 4  8 12]]
[[ 2 10]
 [ 3 11]]
(2, 2)
```

```
a = np.arange(1, 17).reshape(4, 4)
print(a[1:3, 0:2].shape)
print(a[:, -1].shape)
```

```
(2, 2)
(4,)
```

```
a = np.arange(1, 13).reshape(3, 4)
print(a.transpose().shape)
```

```
(4, 3)
```

```
a = np.array([2, 4, 6], dtype=np.int32)
b = a.astype(np.float64)
print(b.dtype)
print(a.dtype)
print(b)
```

```
float64
int32
[2. 4. 6.]
```

```
a = np.array([1, 2, 3], dtype=np.int32)
b = a + 0.5
print(b)
print(b.dtype)
```

```
[1.5 2.5 3.5]
float64
```

```
a = np.array([3.9, 5.2, -2.8])
print(a.astype(int))
```

```
[ 3  5 -2]
```

```
a = np.array([1, 2, 3])
b = a.astype(float)
b[0] = 9.5
print(a)
print(b)
```

```
[1 2 3]
[9.5 2.  3. ]
```

```
a = np.array([1, 2, 3])
a[0] = 4.8
print(a)
```

```
[4 2 3]
```

```
a = np.array([[2], [4], [6]])
b = np.array([10, 20])
print(a + b)
```

```
[[12 22]
 [14 24]
 [16 26]]
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 100, 1000])
print(a * b)
```

```
[[  10  200 3000]
 [  40  500 6000]]
```

```
a = np.zeros((2, 3))
b = np.array([1, 2])
print(a + b)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_768/2872330593.py in <cell line: 0>()
      1 a = np.zeros((2, 3))
      2 b = np.array([1, 2])
----> 3 print(a + b)

ValueError: operands could not be broadcast together with shapes (2,3) (2,)
```

Next steps: Explain error

```
a = np.array([-4, 9, 16])
print(np.abs(a))
print(np.sqrt(np.abs(a)))
```

```
[ 4  9 16]
[2. 3. 4.]
```

```
a = np.array([2, 4, 6])
b = np.array([[10], [20]])
print(a * b + 1)
```

```
[[ 21  41  61]
 [ 41  81 121]]
```

```
a = np.array([1, 2])
b = np.array([3, 4])

print(np.concatenate((a, b)))
# [1 2 3 4]
```

```
print(np.stack((a, b)))
# [[1 2]
#  [3 4]]
```

```
[1 2 3 4]
[[1 2]
 [3 4]]
```

```
a = np.array([[2, 4, 6],
              [8, 10, 12]])
print(a.sum(axis=1))
print(a.mean(axis=0))
```

```
[12 30]
[5. 7. 9.]
```

```
a = np.array([2, 4, 6, 8, 100])
print(np.mean(a))
print(np.median(a))
```

```
24.0
6.0
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])
print(a.max(axis=0))
print(a.min(axis=1))
```

```
[7 8 9]
[1 4 7]
```

```
a = np.array([[1, 2],
              [3, 4]])
print(a.sum())
print(a.sum(axis=0).shape)
print(a.sum(axis=1).shape)
```

```
10
(2,)
(2,)
```

```
a = np.array([[1, 2],
              [3, 4]])
b = np.array([[10, 20],
              [30, 40]])
print(np.concatenate((a, b), axis=1))
print(np.concatenate((a, b), axis=0).shape)
```

```
[[ 1  2 10 20]
 [ 3  4 30 40]]
(4, 2)
```

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(np.stack((a, b), axis=1))
print(np.stack((a, b), axis=1).shape)
```

```
[[1 4]
 [2 5]
 [3 6]]
(3, 2)
```

```
a = np.array([10, 20, 30, 40, 50, 60])
x, y, z = np.split(a, 3)
print(y)
print(z.shape)
```

```
[30 40]
(2,)
```

✦ Gemini

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([[7, 8],
              [9, 10]])
print(np.concatenate((a, b), axis=
print(np.concatenate((a, b), axis=
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1605/3320284187.py in <cell line: 0>()
      3 b = np.array([[7, 8],
      4               [9, 10]])
----> 5 print(np.concatenate((a, b), axis=0))

ValueError: all the input array dimensions except for the concatenation axis must match exactly, but along dimension 1, the
array at index 0 has size 3 and the array at index 1 has size 2
```

Next steps: Explain error

```
np.random.seed(10)
a = np.random.randint(2, 7, size=5)
print(a)
```

```
[3 6 2 3 5]
```

```
np.random.seed(1)
a = np.random.choice([10, 20, 30], size=4, replace=False)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_1605/613385968.py in <cell line: 0>()
      1 np.random.seed(1)
----> 2 a = np.random.choice([10, 20, 30], size=4, replace=False)

numpy/random/mtrand.pyx in numpy.random.mtrand.RandomState.choice()

ValueError: Cannot take a larger sample than population when 'replace=False'
```

Next steps: Explain error

```
a = np.random.normal(loc=50, scale=5, size=(3, 2))
print(a.shape)
```

```
(3, 2)
```

```
np.random.seed(5)
x = np.random.randint(0, 100)

y = np.random.randint(0, 100)

np.random.seed(5)
z = np.random.randint(0, 100)

print(x == z)
print(x == y)
```

```
True
False
```

```
import numpy as np

a = np.array([10, 20, 30, 40])

b = a[1:3]
c = a[1:3].copy()

b[0] = 99
c[1] = 77

print(a)
print(b)
print(c)
```

```
[10 99 30 40]
[99 30]
[20 77]
```

```
a = np.array([1, 2, 3, 4])
b = a[1:]
c = b.copy()
b[0] = 50
c[1] = 60
print(a)
print(b)
print(c)
```

```
[ 1 50  3  4]
[50  3  4]
[ 2 60  4]
```

```
a = np.array([[1, 2], [3, 4]])
b = a.flatten()
c = a.ravel()
b[0] = 10
c[1] = 20
print(a)
```

```
[[ 1 20]
 [ 3  4]]
```

```
a = np.array([5, 10, 15, 20])
b = a[::2]
print(np.shares_memory(a, b))
```

```
True
```

```
a = np.arange(6)
b = a.reshape(2, 3)
b[0, 1] = 99
print(a)
```

```
[ 0 99  2  3  4  5]
```

```
import numpy as np

a = np.array([10, 20, 30, 40, 50])

# Fancy indexing
b = a[[0, 2, 4]]
b[0] = 99

print(a)
print(b)

# Unique values
c = np.unique([2, 2, 4, 4, 6])
print(c)
```

```
[10 20 30 40 50]
[99 30 50]
[2 4 6]
```

```
a = np.array([5, 10, 15, 20, 25])
b = a[[1, 3]]
b[0] = 99
print(a)
print(b)
```

```
[ 5 10 15 20 25]
[99 20]
```

```
a = np.array([1, 2, 3, 4])
b = (a * 2) + 3
print(b)
```

```
[ 5  7  9 11]
```

```
a = np.array([1, 2, 2, 3, 5])
b = np.array([2, 3, 4, 4])
print(np.intersect1d(a, b))
print(np.setdiff1d(a, b))
```

```
[2 3]
[1 5]
```

```
a = np.array([3, 1, 3, 2, 1, 4])
b = np.unique(a)
```

```
c = b[[0, 2]]
c[0] = 99
print(b)
print(c)
```

```
[1 2 3 4]
[99  3]
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.